

PCS-302 : Data Structures Lab

Sorted Matrix Search | $O(n)$ Linear-Time Search in a Row- and Column-Sorted Matrix

Problem Statement

Given a matrix of size $n \times n$, where every row and every column is sorted in increasing order, design an algorithm and write a program to determine whether a given key element is present in the matrix. The solution must run in linear time, $O(n)$.

Example

Matrix (4x4, rows & columns sorted increasing):

```
10  20  30  40
15  25  35  45
27  29  37  48
32  33  39  50
```

Key = 29 --> Found at row 2, col 1

Key = 100 --> Not Found

Algorithm (Staircase Search)

1. Start at the top-right corner: $row = 0, col = n - 1$
2. While $row < n$ AND $col \geq 0$:
 - a. If $mat[row][col] == key$:
 $return (row, col)$ // element found
 - b. Else if $mat[row][col] > key$:
 $col = col - 1$ // move one column left
 - c. Else:
 $row = row + 1$ // move one row down
3. If the loop ends without a match, return NOT FOUND

Time Complexity : $O(n)$ -- each comparison eliminates either one full row or one full column.

Space Complexity: $O(1)$

C Program

```
#include <stdio.h>
#define N 4

int main() {
    int mat[N][N] = {
        {10, 20, 30, 40},
        {15, 25, 35, 45},
        {27, 29, 37, 48},
        {32, 33, 39, 50}
    };

    int key, row, col;
```

```

int found = 0;    // 0 = not found, 1 = found

printf("Enter key to search: ");
scanf("%d", &key);

// Start at the top-right corner
for (row = 0, col = N - 1; row < N && col >= 0; ) {
    if (mat[row][col] == key) {
        found = 1;
        break;                // key found, stop searching
    }
    else if (mat[row][col] > key) {
        col--;                // too big -> move left
    }
    else {
        row++;                // too small -> move down
    }
}

if (found == 1)
    printf("Element %d found at position (%d, %d)\n", key, row, col);
else
    printf("Element %d not found in the matrix\n", key);

return 0;
}

```